

0.1 Introduction

Everything the framework can do, up to this point, it does in code. The analyses of Chapters 4 to 8 are invoked from a Julia session that loads a network, attaches inputs, and calls the modules, and Chapter 9 packages exactly that experience for a programmer. The people whose decisions these analyses serve are mostly not programmers. An asset engineer or a safety analyst understands the system, the failure modes, and what a reliability bound means for an intervention, without wanting to write a line of Julia to obtain one, and the gap between algorithmic availability and practical adoption is documented well enough to have a name, the last mile problem [1, 2].

Existing tools close parts of the gap and leave the whole open. Network visualisation environments such as Gephi [3] and Cytoscape [4] make graphs visual and explorable, and carry no quantitative reliability analysis, so they end where the questions of this thesis begin. The analytical capability lives in libraries, in Matlab [5], R [6], Julia [7], and Python [8], and a library is exactly the delivery form the excluded users cannot consume, because using one means writing code against it. A complete assessment of one system therefore tends to mean several tools, each with its own data format, stitched together by hand, and the stitching is itself a technical barrier.

Delivery architecture raises a separate obstacle. The accessible form of modern software is the web application, and the usual web application is a cloud service, which is precisely what industries with proprietary system data cannot accept. Sending a safety-critical network model to an external service raises custody and security questions [9] that in domains such as nuclear decommissioning are simply disqualifying. The local-first movement argues that software can keep the conveniences of the web while the user's data never leaves the user's machine [10], and that argument fits this framework's audience exactly.

The front end is the framework's answer to both obstacles at once, a no-code web interface that runs entirely on the user's machine. It puts visual network modelling and every analysis of the preceding chapters, with their native uncertainty forms, behind a browser interface that asks the user for files and clicks rather than code, while the computation and the data stay local. This chapter owns the architecture, the contract between the interface and the framework, the data-sovereignty commitment, and the design of the user experience. Chapter 11 then exercises the finished interface end to end on one network.

0.2 A Local Client and a Local Server

The front end is built as a client and a server that happen to share a machine (Figure 1). The server side is an analysis service, a Julia process that hosts the framework of Chap-

ter 9 and exposes its analyses as HTTP endpoints bound to the local machine. The client side is a browser application, written in Angular with the network drawing done in d3, that drives those endpoints and renders their results. The two communicate over HTTP on localhost, so from the user’s seat the whole system is one tool opened in a browser tab, and no traffic leaves the machine.

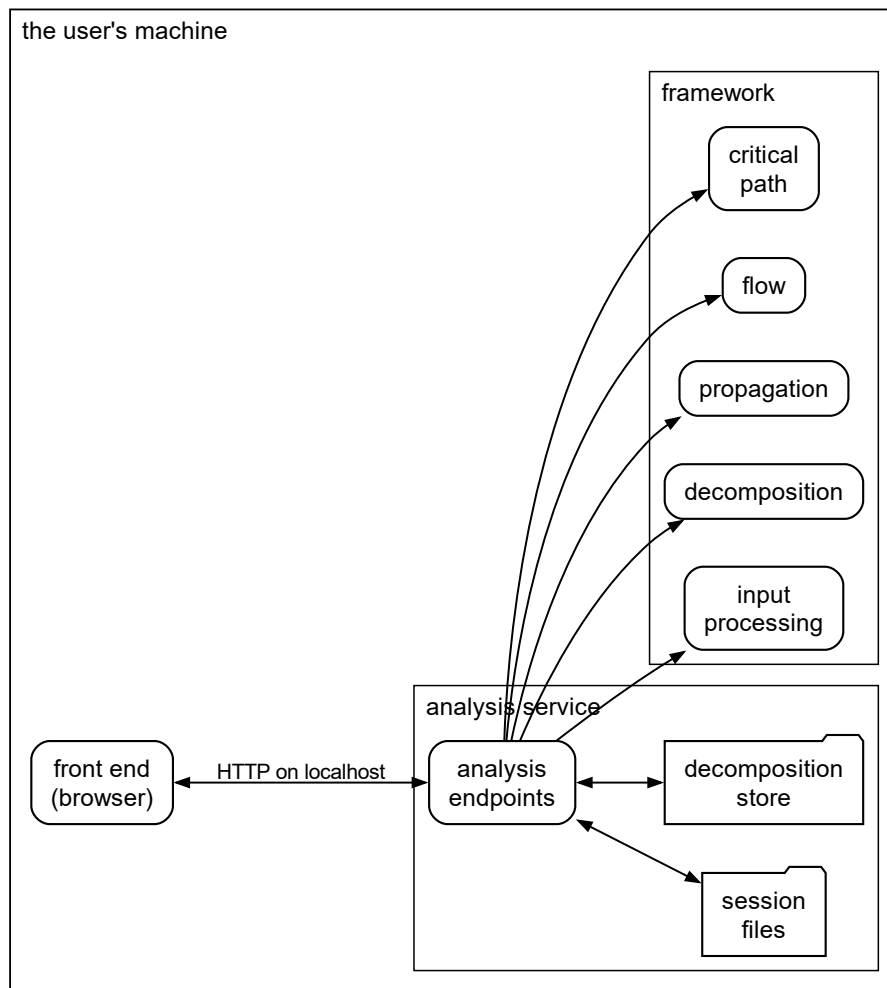


Figure 1: The local-first architecture. The browser front end drives the analysis service over HTTP on the local machine, and the service invokes the framework’s modules, keeping sessions and the decomposition store as files beside the user’s own.

Splitting a single-machine tool into a client and a server is a deliberate trade. The browser brings mature rendering, interaction, and accessibility machinery that no numerical language ecosystem matches, while Julia keeps the computation next to the package that implements it, so each side lives where it is strongest. The two sides can be developed, tested, and replaced independently, and because the boundary between them is ordinary HTTP, the same endpoints that serve the visual interface serve any scripted

client, which keeps the service useful to programmers without a second code path. The boundary itself is written down as a machine-readable interface contract specifying every endpoint, request, and response shape, and both sides are built against that document rather than against each other's internals, which is what allows the interface and the framework to evolve at their own pace without silent drift.

0.3 The Contract

The service exposes one endpoint per analysis class, mirroring the module boundaries of the framework: network structure, diamond decomposition, probability propagation, flow, and critical path, alongside the housekeeping endpoints that manage uploads and sessions. An analysis request does not carry the network embedded in it. It names the user's files, and the service reads them through the input processing of Chapter 4, so the wire format for every input is the file format that chapter already defined, the edge list or adjacency file for structure and the JSON contracts for analysis inputs. The interface invents no second format, the files a user uploads are byte for byte the files the package reads, and the value-form dispatch is the same too, with the service detecting deterministic, interval, or probability-box inputs from the declared data type exactly as input processing does. One convenience softens the file requirements rather than the contract. When analysis inputs arrive without a structure file, the service reconstructs the edge list from the edge keys the inputs themselves carry, so a self-consistent set of analysis files is enough to work with.

Results travel back as JSON keyed by node and edge, and a value never loses its form in transit. A deterministic belief crosses the boundary as a number, an interval as its pair of bounds, and a probability box as a typed summary carrying its bounding curves and moments at the stated discretisation level, so the client always knows what kind of quantity it is rendering and cannot silently collapse an uncertain result into a point. Each analysis endpoint accepts the options its module defines and nothing else. The flow endpoint takes the solver choice among the three of Chapter 7, the failure-combination size, the enumeration limits, the target service level, and degradation scenarios, and its inputs are deterministic capacities because that is the capacity contract of Chapter 4. The critical path endpoint takes the value maps and runs the analyses of Chapter 8, and the propagation endpoint accepts any of the three value forms and can return the decomposition detail alongside the beliefs when the user wants to see the structure behind a number.

Work the user has already paid for is kept. A session is nothing more than a folder holding the uploaded files and a plain JSON record of what has been computed, so analyses persist across visits, a session can be reopened, and deleting one is deleting a folder. The decomposition structure of Chapter 5 gets the same treatment at a finer grain,

because identification is the expensive preprocessing step that every propagation reuses. The service computes the diamond store once, holds it in memory, and persists it beside the session, keyed by the content of the network and prior files so that editing an input invalidates exactly the store it affects. A propagation request that follows a decomposition request, or a second propagation under a different scenario, finds the structure already in place. The store that Chapter 5 built to keep the algorithms from repeating themselves here keeps the user’s sessions from repeating themselves.

0.4 Data Sovereignty

Nothing described above leaves the user’s machine, and that is a design commitment rather than a deployment accident. There is no account to create, no telemetry, and no remote service in the path. The browser talks to a process on the same computer, and every artefact of a session, the uploaded network, the analysis inputs, the computed results, and the cached decomposition, exists as ordinary files in one folder that the user can inspect, archive to controlled storage, or destroy, with nothing held anywhere else. The web technologies are used for what they are good at, rendering and interaction, without importing the custody model that usually comes with them.

For the industries this framework grew out of, the commitment is not a nicety. A decommissioning plan’s dependency network, a defence logistics topology, or a utility’s failure data are exactly the models an organisation cannot place in external custody, whatever the convenience [9], and tools that require it simply do not get adopted there. Local-first software keeps the usability argument and drops the custody problem [10]. For this framework’s audience the practical consequence is plain enough to state in one line. Adopting the interface requires no security review of a third party, because there is no third party.

0.5 The Interface

The interface arranges the framework’s capabilities as a guided pipeline (Figure 2), and each step unlocks when the data it needs exists. A user cannot reach a propagation view before a network is loaded, or a comparison view before there are results to compare, so the interface never presents a dead control, and a first-time user is steered through the same order this thesis follows, structure first, then analysis, then comparison. Familiar users are not forced through ceremony, since any unlocked step is reachable directly, and the interface preserves each view’s state, its filters, selections, and open tabs, across navigation, so moving between views to cross-reference results costs nothing.

A network enters as files, either picked individually or as a whole folder that the interface sorts by naming convention into the structure file and the analysis inputs. The

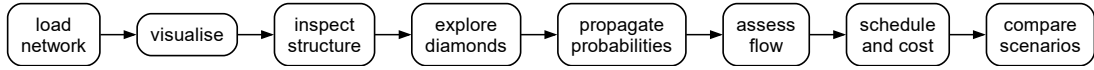


Figure 2: The guided pipeline. Each step unlocks when its prerequisite data exists, and any unlocked step is reachable directly.

same convention carries scenarios, since a network folder may hold several input sets, one per value form or operating case, and the interface treats each as a scenario of the same network, so one upload supports the whole comparative workflow. What the user then sees first is the network itself (Figure 3). The drawing is built from the unified graph object, laid out layer by layer along the iteration sets of Chapter 4 so that information visibly flows down the page in the order the algorithms process it, with the node roles of the model chapter, sources, sinks, forks, and joins, distinguished at a glance, and a force-directed alternative available when the layered view is not the natural reading. A structure dashboard beside it reports what input processing derived, the roles, the layers, and the connectivity, which is frequently where a modelling error in the input files announces itself, before any analysis has been run.

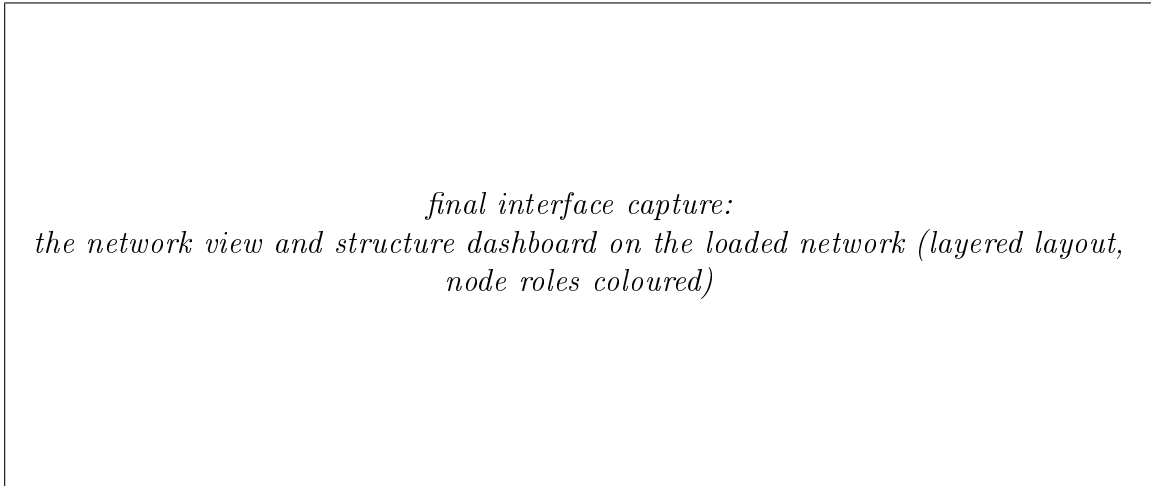


Figure 3: The network as the interface presents it, the interactive layered drawing and the structure dashboard derived by input processing.

The decomposition of Chapter 5 is not hidden inside the propagation. The diamonds view lists the identified structures, and each diamond opens into a detail view that exploits the self-similarity that chapter established. A stored diamond is a graph object of the same form as the network, so the interface can present one as a network in its own right and run any analysis on it in isolation, with its local sources overridable, which turns the decomposition from an internal accelerator into an instrument for localised investigation. A reconvergence that dominates the network’s uncertainty can be studied on its own, at its own scale, without leaving the tool.

The analysis views deliver the toolkits of Chapters 6 to 8, and their common design rule is honesty about the form of a value. A propagated belief under a deterministic scenario is shown as a number, under an interval scenario as a range, and under a probability-box scenario as a band, with width statistics summarising where the network’s uncertainty concentrates, and at no point does the interface average, midpoint, or otherwise flatten an uncertain result into a false precision (Figure 4). The flow views report the throughput, bottleneck, sensitivity, and threshold results of Chapter 7, the scheduling views the time and cost analyses of Chapter 8, and a cross-scenario profile view sets the scenarios of one network side by side, which is where the comparative questions, what the interval inputs do to the answer, which design carries the risk, get answered without exporting anything to a spreadsheet. In-app documentation explains each view and the input formats in the user’s terms, so the interface carries its own manual.

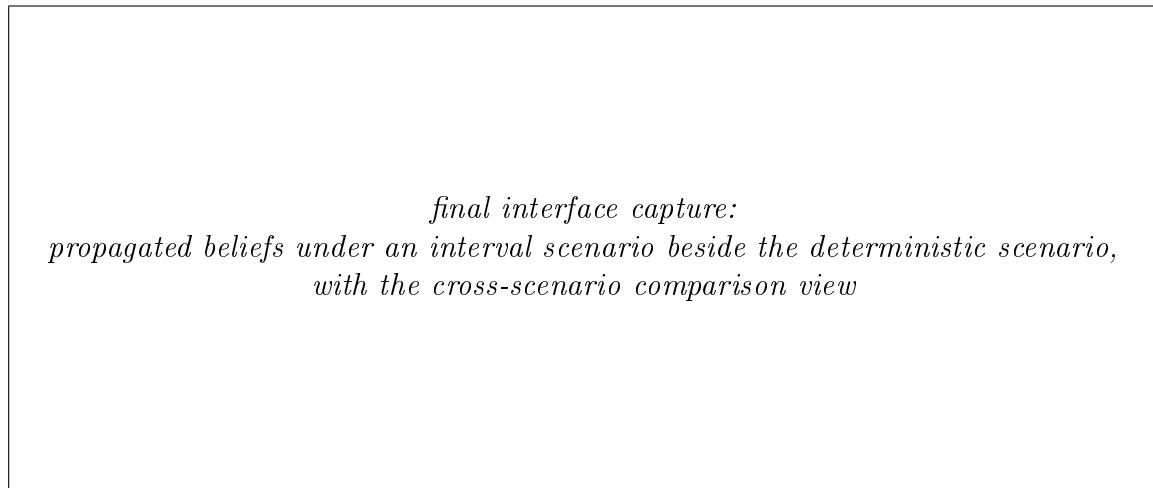


Figure 4: Propagation results as the interface presents them, ranges shown as ranges, with scenarios of the same network compared side by side.

The boundaries of the design are as deliberate as its features. The interface is a single-user local tool, with no collaboration layer, because its custody commitment is the point. It exposes the framework’s analyses and does not author new ones, since extending the framework is a programmer’s task and belongs to the package of Chapter 9. And it adds no analytical capability of its own, because everything it shows is produced by the modules the preceding chapters validated, which is precisely what makes its results trustworthy: the interface is a window, not a second implementation.

0.6 Summary

The front end closes the framework’s last mile. A local analysis service exposes the modules of Chapters 4 to 8 behind a documented contract in which the user’s files are the currency, inputs keep the formats input processing defined, results keep their value

forms, and paid-for work persists in sessions and a content-keyed decomposition store. A browser front end turns that contract into a guided, no-code workflow, from upload through structure, decomposition, and the three toolkits to cross-scenario comparison, rendering uncertain results as the ranges and bands they are. Everything runs on the user's machine and every artefact is an ordinary file the user owns, which is what makes the tool adoptable in the proprietary-data industries the framework serves. Chapter 11 puts the whole interface to work on a single network, end to end.

Bibliography

- [1] J. E. Hannay, C. MacLeod, J. Singer, H. P. Langtangen, D. Pfahl, and G. Wilson, “How do scientists develop and use scientific software?” In *Proceedings of the 2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, IEEE, 2009, pp. 1–8. DOI: 10.1109/SECSE.2009.5069155
- [2] P. Prabhu et al., “A survey of the practice of computational science,” in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11, New York, NY: ACM, 2011, pp. 1–12. DOI: 10.1145/2063348.2063374
- [3] M. Bastian, S. Heymann, and M. Jacomy, “Gephi: An open source software for exploring and manipulating networks,” in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, 2009, pp. 361–362. DOI: 10.1609/icwsm.v3i1.13937
- [4] P. Shannon et al., “Cytoscape: A software environment for integrated models of biomolecular interaction networks,” *Genome Research*, vol. 13, no. 11, pp. 2498–2504, 2003. DOI: 10.1101/gr.1239303
- [5] E. Patelli, H. George-Williams, J. Sadeghi, R. Rocchetta, M. Broggi, and M. de Angelis, “OpenCossan 2.0: An efficient computational toolbox for risk, reliability and resilience analysis,” in *Proceedings of the Joint ICVRAM ISUMA UNCERTAINTIES Conference*, A. T. Beck, G. F. M. de Souza, and M. A. Trindade, Eds., 2018.
- [6] S. Ferson, *Pba.r: Probability bounds analysis in R*, <https://github.com/ScottFerson/pba.r>, 2019.
- [7] A. Gray, S. Ferson, and E. Patelli, *ProbabilityBoundsAnalysis.jl: Arithmetic with sets of distributions*, Proceedings of JuliaCon, 2021. [Online]. Available: <https://github.com/Institute-for-Risk-and-Uncertainty/ProbabilityBoundsAnalysis.jl>
- [8] N. Gray, S. Ferson, M. De Angelis, A. Gray, and F. Baumont de Oliveira, “Probability bounds analysis for Python,” *Software Impacts*, vol. 12, p. 100 246, 2022. DOI: 10.1016/j.simpa.2022.100246

- [9] K. Hashizume, D. G. Rosado, E. Fernández-Medina, and E. B. Fernández, “An analysis of security issues for cloud computing,” *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1–13, 2013. DOI: 10.1186/1869-0238-4-5
- [10] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, ser. Onward! 2019, ACM, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737